Advanced Database (CSC-461)
B.Sc.CSIT/VIII Semester

Course Title: Advanced Database **Full Marks:** 60 + 20 + 20
Course No: (CSC-461) **Pass Marks:** 24 + 8 + 8
Nature of the Course: Theory + Lab **Credit Hrs:** 3
Semester: VIII

Course Description:
This course includes advanced concept of database system. The main topics covered are advanced concept of relational data model, Extended E-R model, new database management technologies, query optimization, NoSQL database and big data processing techniques.

Course Objectives:
At the end of the course students should be able to know new developments in database technology, interpret and explain the impact of emerging database standards, evaluate the contribution of database theory to practical implementations of database management systems. Also, students should be able to develop more advanced application using MapReduce and Hadoop.

Course Contents:

Unit 1: Enhanced Entity Relationship Model and Relational Model (8 Hrs.)
Entity Relationship Model Revised; Subclasses, Superclasses and Inheritance; Specialization and Generalization; Constraints and characteristics of specialization and Generalization; Union Types; Aggregation; Relational Model Revised; Converting ER and EER Model to Relational Model; SQL and Advanced Features; Concepts of File Structures, Hashing, and Indexing

Unit 2: Object and Object Relational Databases (10 Hrs.)
Object Database Concepts; Object Database Extensions to SQL; The ODMG Object Model and the Object Definition Language ODL; Object Database Conceptual Design; Object Query Language OQL; Language Binding in the ODMG Standard

Unit 3: Query Processing and Optimization (7 Hrs.)
Concept of Query Processing; Query Trees and Heuristics for Query Optimization; Choice of Query Execution Plans; Cost-Based Optimization

Unit 4: Distributed Databases, NOSQL Systems, and BigData (12 Hrs.)
Distributed Database Concepts and Advantages; Data Fragmentation, Replication and Allocation Techniques for Distributed Database Design; Types of Distributed Database Systems; Distributed Database Architectures

Introduction to NOSQL Systems; The CAP Theorem; Document-based, Key-value Stores, Column-based, and Graph-based Systems; BigData; MapReduce; Hadoop

Unit 5: Advanced Database Models, Systems, and Applications (8 Hrs.)
Active Database Concepts and Triggers; Temporal Database Concepts; Spatial Database Concepts; Multimedia Database Concepts; Deductive Database Concepts; Introduction to Information Retrieval and Web Search

Laboratory Works:
Students should implement different concepts of database system studied in each unit of the course during lab time and should submit a mini project at the end the course.

Recommended Books:

1. Elmasri and Navathe, Fundamentals of Database Systems, Pearson Education.
2. Raghu Ramakrishnan, Johannes Gehrke, Database Management Systems, McGraw-Hill
3. Korth, Silberchatz, Sudarshan, Database System Concepts, McGraw-Hill.
4. Peter Rob and Coronel, Database Systems, Design, Implementation and Management, Thomson Learning.
5. C. J. Date & Longman, Introduction to Database Systems, Pearson Education 6. Tiwari, Shashank and Safari, professional Nosql, O'Reilly Media Company.
7. Gunarathne, Thilina Hadoop MapReduce v2 Cookbook: Explore the Hadoop MapReduce v2.
8. Ecosystem to Gain Insights from very Large Datasets, 2nd Edition, PACKT Publishing.

Unit 1: Enhanced Entity Relationship Model and Relational Model

Preliminary Study

Data

- The collection of raw facts is called data.
- Data is a value that also represents the description of an object or event
- Data is more historical, while information has a higher level, is more dynamic, and has very important value.
- Eg. RAM, 123, Dell, 5, image etc.

Information

- Information is the result of processing data in a form that is more useful and meaningful to the recipient, that describes a real event (*fact*) used for decision-making.
- Eg. Rs.5, RAM Memory, Dell Computer, Elon Musk image, etc.

File

- A file is a collection of related data stored together in a database.
- It holds multiple records of the same type.
- Each file usually represents a table or dataset.
- Example: A *Student File* contains all student records.

Records

- A file may be further divided into more descriptive subdivisions, called records. In other words, a record is a collection of related data items as a single unit.
- A record is a complete set of related fields.
- It represents one instance or entry in a table.
- In a table, each row is a record.
- In a practical way, one complete row.
- Example: A record may store one student's details.

Tuples

- The row of a table is called tuples. It is also called value of a table.
- A tuple is another term for a record in relational databases.
- It represents one row in a relational table.
- In the theoretical/relational model set of attributes in a row.
- Each tuple holds specific values for all attributes.
- Example: (101, "Anita Sharma", "Kathmandu", "A+").

Fields

- A field is the smallest unit of data in a database.
- It represents a single attribute or property of an entity.
- In a table, fields appear as columns.
- Example: *Name, Roll No.,* and *Address* are fields in a student table.

Database

- Database is a collection of connected data stored together on a medium, organized according to a specific scheme or structure, and with software to perform manipulations for specific uses.
- A database is a collection of related data necessary to manage an organization. By data, we mean known facts that can be recorded and have implicit meaning. For example, consider names, telephone numbers and addresses of the people. We may have recorded this data in an indexed address book, or we may have recorded on the hard drive, using a personal computer and software such as MS-Access, or Excel. This is the collection of related data with an implicit meaning and hence is a database. A database is logically coherent collection of data with some inherent meaning. A database is designed, built and populated with data for specific purpose. It excludes transient data such as: input documents, reports and intermediate results obtained during processing.

- **Basic database operations:**
  - Create database
  - Drop database
  - DISPLAY
  - SORT
  - UPDATE
  - Create table
  - Drop table
  - Insert
  - Retrieve / Search
  - Update
  - Delete, etc.

DBMS

- A Database Management System (DBMS) is software designed to store, organize, and manage databases efficiently.
- It acts as an interface between the database and application programs, allowing users to access data in the required form.
- A DBMS ensures data security, accuracy, consistency, and shared access among multiple users.
- It provides tools and procedures to define, construct, manipulate, and share data within an organization.
- The system ensures that the required data is available in the desired form for various applications across different departments.
- In simple terms, a DBMS is a general-purpose software system that manages and controls databases.
- The main objective of a DBMS is to store and manage data conveniently and efficiently.
- Figure 1 illustrates the distinction between a Database (a structured collection of data) and a Database Management System (the software used to manage that data).

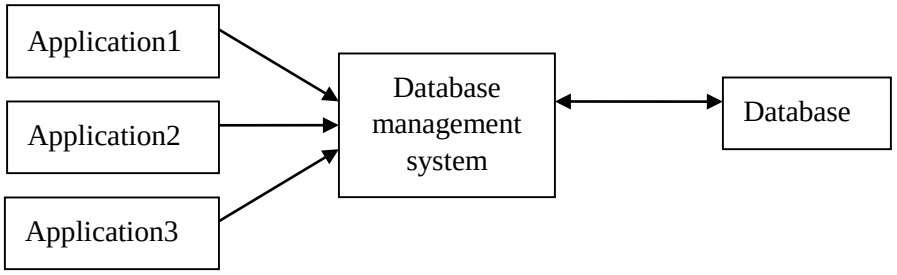


Figure 1: illustrates the distinction between a DBMS and a Database.

Why use DBMS

- DBMS is required for:
    a. Data independence and efficient access
    b. Reduce app development time
    c. Data integrity and security
    d. Data uniformity administration
    e. Concurrent access and crash fixes

Objectives of DBMS

1. Data Sharing and Accessibility
    - The main objective of a DBMS is to enable efficient and secure sharing of data among multiple users and applications.
    - It allows simultaneous access to the same data without conflicts, ensuring consistency across departments.
    - Modern DBMSs support distributed databases and cloud-based systems, making data available anytime and anywhere with proper authorization.
2. Data Integrity and Accuracy
    - Data integrity ensures that information stored in the database remains accurate, valid, and consistent over time.
    - Integrity constraints, validation rules, and referential integrity mechanisms help prevent errors caused by incorrect input, hardware failure, or system crashes.
    - Advanced DBMSs automatically enforce data validation and error-checking procedures to maintain reliability and trustworthiness.
3. Data Independence
    - Data independence separates the physical storage of data from how it is accessed by applications.
    - This ensures that any modification to the database structure (e.g., adding new fields or changing storage format) does not affect existing applications.
    - Modern DBMSs achieve both logical and physical data independence, improving flexibility and scalability.
4. Minimization of Data Redundancy
    - A DBMS is designed to eliminate unnecessary duplication of data.
    - By storing data in a normalized structure, it reduces redundancy and prevents inconsistencies.
    - In specific cases, controlled redundancy may be introduced to improve performance or ensure data availability in distributed systems.
5. Data Security and Privacy
    - Data security is a key objective of any modern DBMS.

- It involves implementing authentication, authorization, encryption, and role-based access control to prevent unauthorized access or data breaches.
- With growing privacy concerns, DBMSs now comply with data protection laws (such as GDPR) and support auditing and monitoring for accountability.

6. Data Maintenance and Quality Management
   - A well-managed DBMS provides tools for updating, modifying, and deleting records to keep information current and relevant.
   - Modern systems include automated maintenance features, such as scheduled backups, data cleaning, and synchronization with external sources.
   - These features ensure high data quality and long-term system efficiency.
7. Management and Administrative Control
   - As data becomes a strategic asset, strong management control over database operations is essential.
   - Database administrators (DBAs) monitor performance, manage user access, and ensure compliance with organizational policies.
   - Modern DBMSs provide dashboard-based management tools**,** performance tuning**,** and automated recovery mechanisms for enhanced oversight and reliability.

Example uses of DBMS

- Airlines – reservations and schedules etc.
- Banking – customer information and accounts etc.
- Universities – student information, grades etc.
- Government – taxes, budgets, census etc.
- Sales – inventory, customer info etc.
- Credit cards – transactions, customer info etc.
- Newspapers – track subscribers, advertising revenue.
- Financial – stock prices, portfolio info.
- Telecommunications – records of calls made.

When a DBMS is Needed

- When large volumes of interrelated data must be integrated, managed, and accessed efficiently.
- For applications requiring data sharing, concurrent access, and collaboration across departments or systems.
- When there is a need for advanced data analytics, such as data mining**,** deductive databases, or active databases that respond to events automatically.
- To ensure high Quality of Service (QoS) through data consistency, security, scalability, and reliability.
- In environments that demand real-time access**,** cloud storage, or distributed data management across multiple servers.

Advantages of the DBMS Approach

- Controlled Data Redundancy
    - A DBMS minimizes unnecessary duplication of data through normalization and centralized storage.
    - This ensures consistency and saves storage space.
- Restricted Unauthorized Access
    - Modern DBMSs include strong security mechanisms such as authentication, authorization, role-based access control, and encryption.
    - These protect sensitive data from unauthorized access and cyber threats.
- Persistent Storage for Application Data
    - DBMS provides a permanent and reliable storage mechanism for program objects and data structures.
    - This ensures data remains intact even after application or system failures.
- Efficient Query Processing
    - Advanced indexing, caching, and query optimization techniques allow for fast and efficient data retrieval**.**

    - DBMSs now support SQL and NoSQL querying**,** in-memory computing**,** and AI-assisted query tuning**.**
- Backup and Recovery Management
    - DBMSs automatically perform data backup**,** replication, and recovery operations to protect data against system crashes or power failures.
    - Cloud-based DBMS platforms often include real-time disaster recovery features.
- Multiple User Interfaces
    - DBMSs support various interfaces such as graphical dashboards, APIs, command-line tools, and web-based access**.**
    - This makes it easier for administrators, developers, and analysts to interact with the data.
- Representation of Complex Relationships
    - Modern relational and object-oriented DBMSs can represent complex data relationships and constraints.
    - They also support hierarchical, network, and graph-based data models used in social networks, recommendation systems, and knowledge graphs.
- Enforcement of Integrity Constraints
    - DBMSs automatically enforce data integrity rules, ensuring accuracy, validity, and consistency.
    - Constraints such as primary keys, foreign keys, and unique attributes prevent data anomalies.

Disadvantages of the DBMS Approach

- High Initial Cost
    - Commercial DBMS software can be expensive to purchase or license, especially for enterprise-scale systems.
    - Even open-source systems require investment in setup and configuration.
- Maintenance and Administration Overhead
    - Requires skilled database administrators (DBAs) for performance tuning, security management, and backup operations.

    - Continuous monitoring and updates increase long-term costs.
- Resource Intensive
    - DBMSs often demand high processing power, memory, and storage resources, especially when handling large datasets or complex queries.
    - This may not be practical for small-scale or simple applications.
- Overkill for Small Applications
    - For small databases with simple structures and limited users, a full DBMS may be unnecessary.
    - Simple file-based systems can sometimes provide faster and cheaper solutions.
- Complexity
    - The configuration and operation of modern DBMSs can be complex, requiring specialized technical expertise.
    - Mismanagement may lead to data loss, performance issues, or security vulnerabilities.

DBMS Languages

1. Data Definition Language (DDL)
    - DDL is used by database administrators (DBAs) and database designers to define the structure of the database.
    - It specifies the conceptual schema, including tables, fields, data types, and relationships.
    - DDL commands are used to create**,** modify, and delete database objects such as tables, views, and indexes.
    - In some systems, Storage Definition Language (SDL) and View Definition Language (VDL) are used separately to define internal and external schemas.
    - Common DDL commands: CREATE, ALTER, DROP, and TRUNCATE.
2. Data Manipulation Language (DML)
    - DML is used to retrieve, insert, update, and delete data in the database.
    - It allows users and programs to interact with the stored data.
    - DML can be of two types:

        - Embedded DML: DML statements are written inside a host programming language like C, Java, or Python**.**
        - Interactive DML (Query Language)**:** DML statements are executed directly by users (e.g., SQL queries).
    - Common DML commands: SELECT, INSERT, UPDATE, and DELETE.
3. High-Level or Non-Procedural Languages
    - These languages allow users to specify what data to retrieve, not how to retrieve it.
    - They are set-oriented and easier to use for complex queries.
    - SQL (Structured Query Language) is the most common example.
    - Such languages are also known as declarative languages**.**
4. Low-Level or Procedural Languages
    - These languages describe how to retrieve data, often processing one record at a time.
    - They provide looping, conditional, and control structures to manage data step by step.
    - Commonly used in embedded or application-level programming where detailed data control is needed.
    - Examples include PL/SQL**,** T-SQL, and procedural extensions of SQL.

View of data

The main purpose of a database is to provide users with an abstract view of the data, i.e. the system hides certain details of how the data are stored and maintained. This is called data abstraction.

Three level architecture (ANSI / SPARC Architecture)

- View level.
- Logical / conceptual level.
- Physical level.

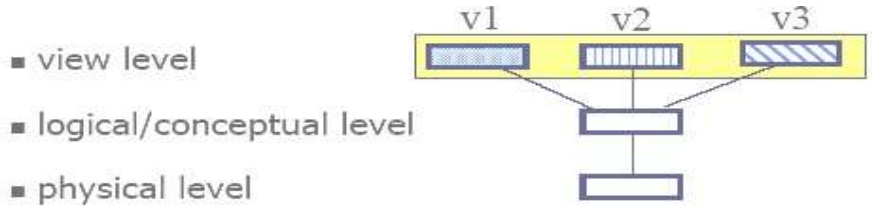


Figure 2: Three level architecture view

1. View level (external level)

It is the highest level of data abstraction. It describes only part of the entire database. Different users may view the data in different ways. This may involve viewing data in different formats or derived values which are not stored persistently in the database only the data relevant to the particular user need be included in the user's view. eg .CS majors, math major.

2. Logical level (conceptual level);

It describes what data are stored in database and relationship among these data. The entire database schema is described in terms of small number of relatively small structures. The logical structures of the entire database described by the conceptual schema. It includes all data entities attributes and relationship, integrity constrains are defined. All external view are derivable from conceptual schema it is independent of any storage details e.g.

Course (course no, course name, credits, dept)

Student (student id Lname, Fname, class, maior)

Grade-report (students ID, course No, Grade, tem

3. Physical level (Internal level)

It is the lower level of abstraction, describes how data are stored. It describes the physical storage characteristics of the data. Such as storage space allocation, indexes, It is independent of any particular application software. It also describes how the tables are stored, how many bytes / attributes it allocated etc.

Course Content (Chapter 1) Start from Here:

The Entity Relationship (E-R) model

- E-R Model represents the real-world structure using entities (objects) and the relationships between them.
- It provides a conceptual framework for describing the data and their relationships in a database system.
- The E-R model focuses on modeling the logical view of data, independent of physical storage details.
- The main components of the E-R model include Entity, Entity Type, and Entity Set.

Entity-Relationship (E-R) diagram

- E-R Diagram is a graphical representation of entities, their attributes, and the relationships among them.
- It visually depicts the overall logical structure of a database system.
- The E-R diagram helps in conceptual database design, providing a clear and organized view of data and its interconnections.
- It was developed to simplify and improve the database design process through its clarity and easy-to-understand visual representation.

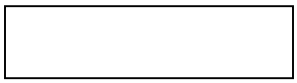
The basic components of an E-R diagram include:

- Entities – Real-world objects or concepts represented as rectangles.
- Attributes – Properties or characteristics of entities, shown as ovals.
- Relationships – Associations among entities, represented as diamonds.
- Links – Lines connecting entities, attributes, and relationships to show their connections.

Entity:

- An entity is a “thing” or “object” in the real word that is distinguishable from other objects.
- An object with a physical existence like, particular person, car, house, employ, company, etc.
- For example each person is an entity. An entity has a set of properties and the values for some set of properties may uniquely identify an entity.

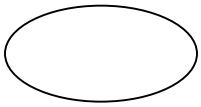
- For example, if student is an entity, is identified by registration number.
- It is represented by rectangle.



Entity

Attribute:

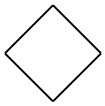
- Attributes are properties possessed by an entity or relationship.
- An entity is represented by the set of characteristics called attributes.
- Each attribute has set of values called domain.
- Eg. Possible entity of loan entity set has loan number and loan amount, similarly, the domain of attribute might be a set of positive integers.



Attribute

Relationship:

- A relationship is an association among several entities and represents meaningful dependencies between them.
- Eg. the association between teachers and students is teaching.
- It is represented by a diamond.



Relationship

Strong and Weak Entity:

- An entity set may not have sufficient attributes to form a primary key. Such entity set is termed as a weak entity set.
- An entity set that has a primary key is termed as a strong entity set.
- For a weak entity set to be meaningful, it must be associated with another entity set, called the identifying entity set.
- The relationship associating the weak entity set with the identifying entity set is called the identifying relationship.

Symbols used in E-R Diagram:

Symbol	Shape Name	Symbol Description
Entities		
	Entity	An entity is represented by a rectangle which contains the entity's name.
	Weak Entity	An entity that cannot be uniquely identified by its attributes alone. The existence of a weak entity is dependent upon another entity called the owner entity. The weak entity's identifier is a combination of the identifier of the owner entity and the partial key of the weak entity.
	Associative Entity	An entity used in a many-to-many relationship (represents an extra table). All relationships for the associative entity should be many

Figure 4: Entity related symbols

Attributes		
	Attribute	In the Chen notation, each attribute is represented by an oval containing atributte's name
	Key attribute	An attribute that uniquely identifies a particular entity. The name of a key attribute is underscored.
	Multivalued attribute	An attribute that can have many values (there are many distinct values entered for it in the same column of the table). Multivalued attribute is depicted by a dual oval.
	Derived attribute	An attribute whose value is calculated (derived) from other attributes. The derived attribute may or may not be physically stored in the database. In the Chen notation, this attribute is represented by dashed oval.

Figure 3: Attribute related symbols

Relationships		
	Strong relationship	A relationship where entity is existence-independent of other entities, and PK of Child doesn't contain PK component of Parent Entity. A strong relationship is represented by a single rhombus
	Weak (identifying) relationship	A relationship where Child entity is existence-dependent on parent, and PK of Child Entity contains PK component of Parent Entity. This relationship is represented by a double rhombus.

Figure 5 Relationship types symbols

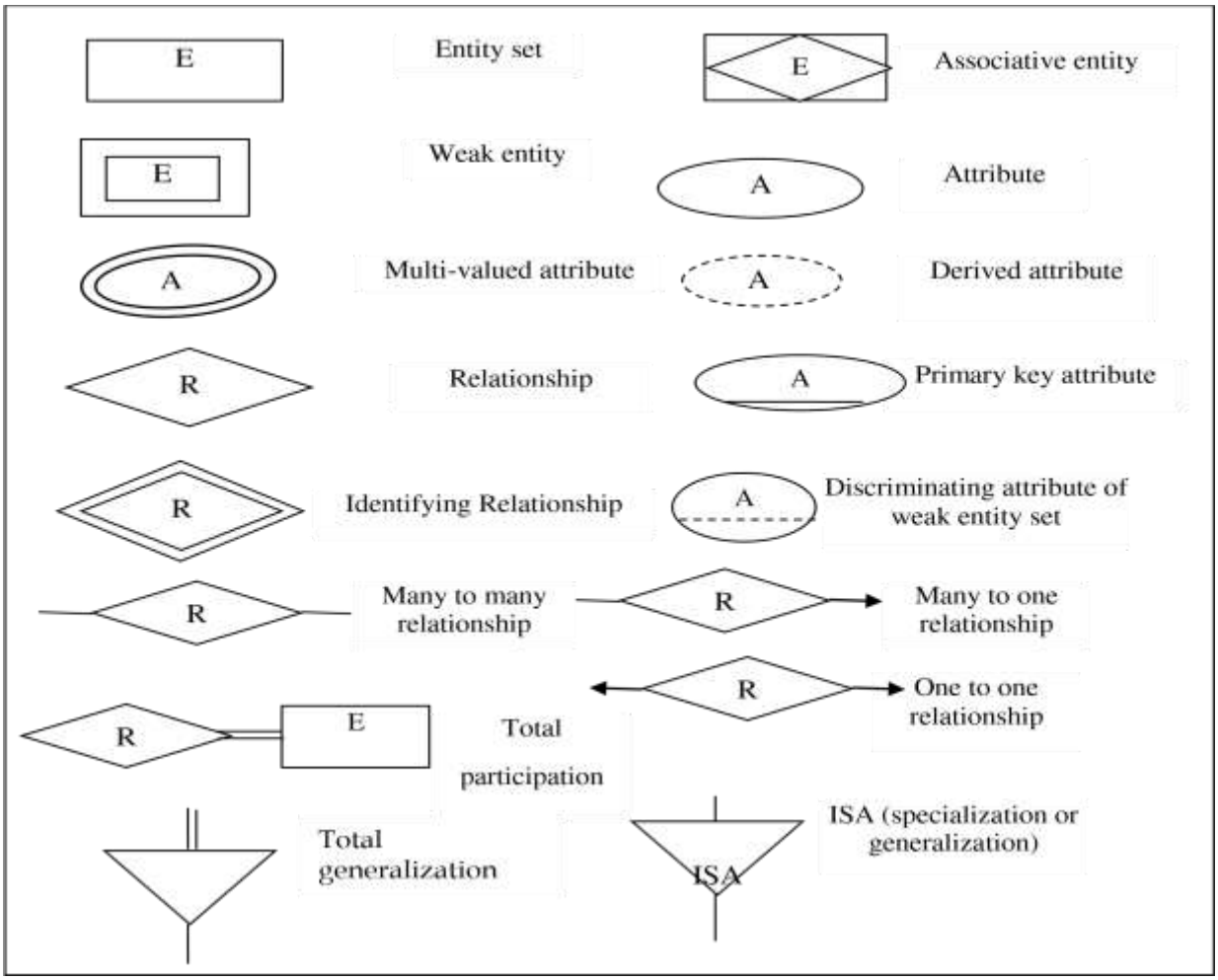


Figure 6 Total Symbols

Database Key:

Set of one or more attributes whose values are distinct for each individual entity in the entity set is called key, and its values can be used to identify each entity uniquely. Types of keys are:

1. Super Key:

- A super key is a set of one or more attributes that allows us to uniquely identify an entity in a set.
- Example: Customer name and social security number together can be a super key for the entity set *Customer*.

2. Candidate Key:

- A super key that does not have any proper subset which is also a super key is called a candidate key.
- Example:
  - *Student_id* is a candidate key for the entity set *Student*.
  - However, the set of attributes {roll_number, name, program, semester, section} is not a candidate key for the entity set *Student*.

3. Primary Key:

- A primary key is a candidate key chosen by the database designer as the principal means of uniquely identifying entities within an entity set.

4. Composite Key:

- If a primary key contains more than one attribute, then it is called a composite key.

5. Foreign Key:

- A set of attributes used to establish a reference to the primary key of another table is called a foreign key.
- It is used to establish a relationship between two tables.

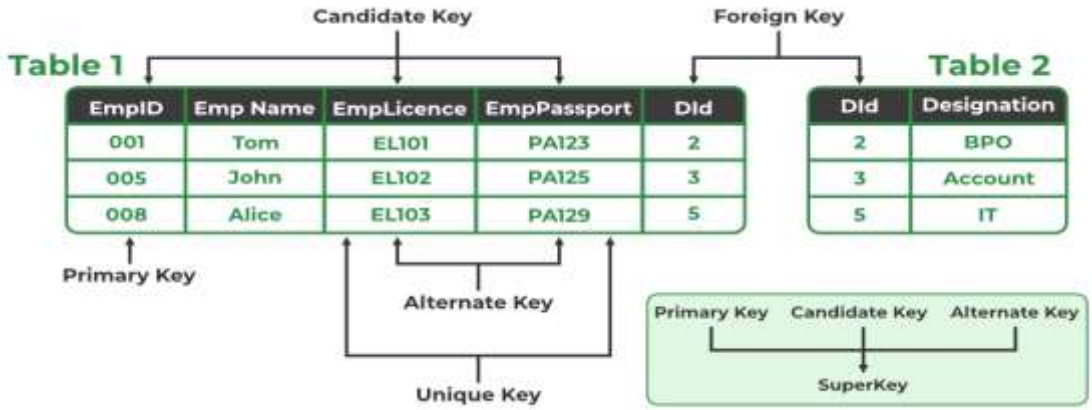


Figure 7: Types of keys

Enhanced or Extended E-R Model (EER Model):

- The ER modeling concepts are not sufficient for representing new database applications, which are more complex than traditional applications.
- To model modern complex systems such as databases for engineering design and manufacturing (CAM/CAD), telecommunications, GIS, etc.
- It is not enough with ER model. Thus, the modeling of such systems can be made easy with the help of an enhanced/extended ER model.
- The enhanced ER model includes all of the concepts introduced by the ER model.
- Additionally, it includes the concepts of a subclass and superclass along with the concept of generalizations and specializations.
- There are basically four concepts of EER model, which are as follows:

1. **Subclass/superclass relationship:**
    - Super-class is an entity type that has a relationship with one or more subtypes.
    - An entity can't exist in a database by being a member of any super-class.
    - For example: shape super-class is having subgroups as super, circle, triangle.
    - A sub-class is a group of entities with unique attributes.
    - Sub-class inherits properties and attributes from its superclass.
    - For example: square, circle, triangle are subclasses are the shape of super-class.

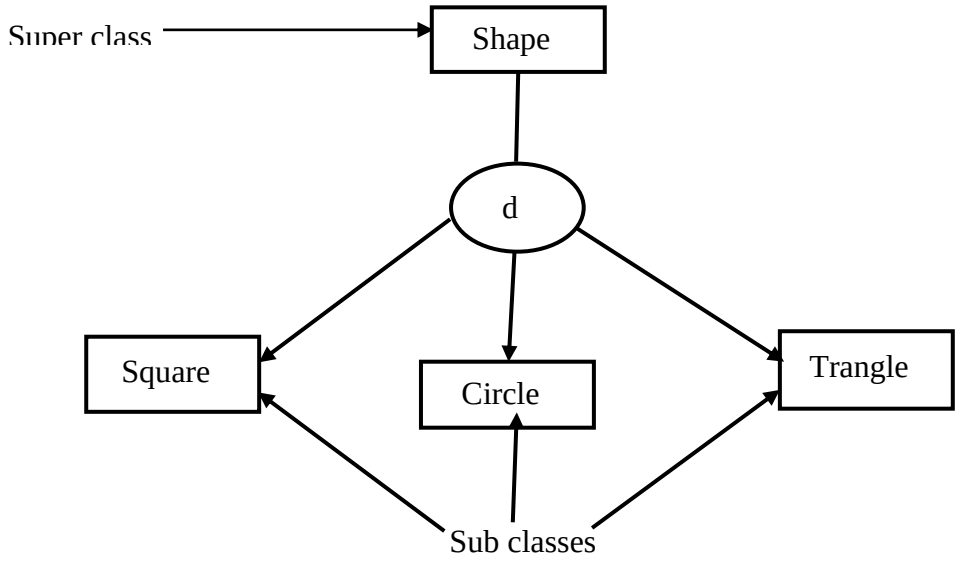


Figure 8 Sub class superclass relationship

2. **Specialization and Generalization:**

- The process of defining a set of subclasses from a superclass is known as specialization.
- The set of subclasses is based upon some distinguishing characteristics of the entities in the superclass.
- It is a top-down approach, in which one higher entity can be broken down into two lower level entity.
- For example: the entity set person can be specialized into entity sets employee, and customer. Further, on the basis of job type the entity set employee can be divided into entity sets manager, teller, typist etc.
- Thus, specialization may create hierarchy.
- Attributes of a subclass are called specific attributes.

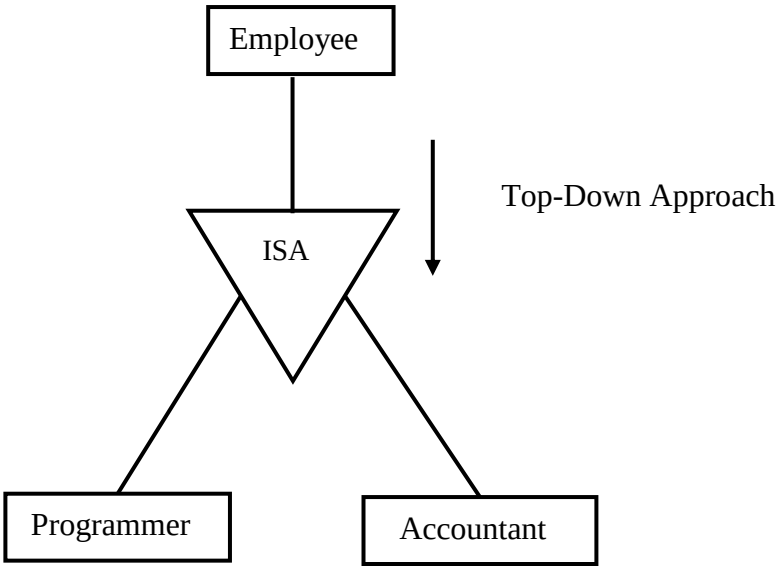


Figure 9 Specialization

- Generalization is the reverse of the specialization.
- Several classes with common features are generalized into a super class; original classes become its subclasses.
- It is a bottom-up design process. Here, we combine a number of entity sets that share the same features into a higher-level entity set.

- Designer applies generalization is to emphasize the similarities among the entity sets and hide their differences.
- Specialization and generalization are simple inversions of each other; they are represented in an E-R diagram in the same way.
- The terms specialization and generalization are used interchangeably.

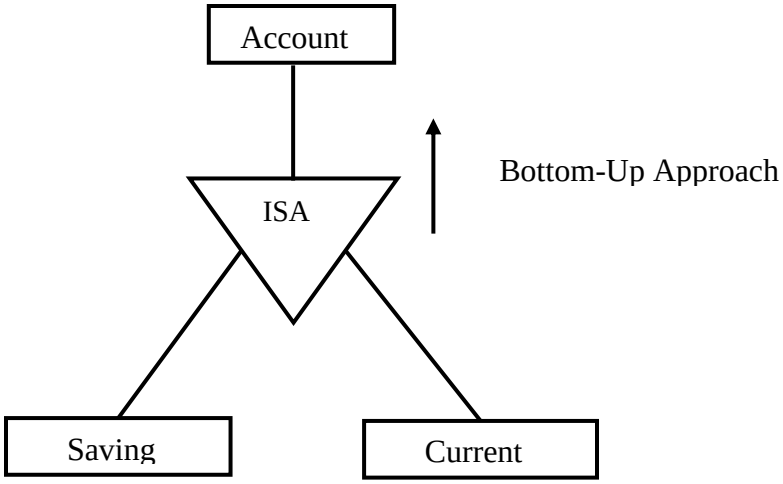


Figure 10 Generalization

3. **Categories (Union Types):**
   - Category represents a single super class or sub class relationship with more than one super class.
   - It is possible that single superclass/subclass relationship has more than one super classes representing different entity types.
   - In this case, the subclass will represent a collection of objects that is the union of distinct entity types; we call such a subclass a union type or a category.
   - A category has two or more super classes that may represent distinct entity types, whereas non-category superclass/subclass relationships always have a single superclass.
   - Here, owner is a subset of the union of the three super classes company, bank and person.

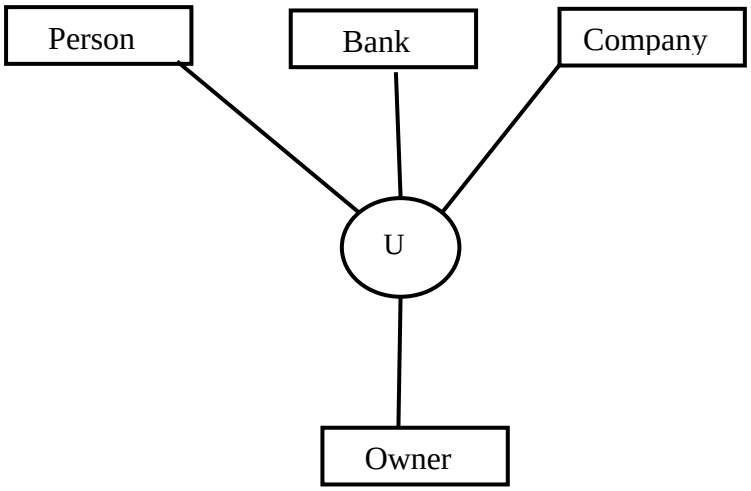


Figure 12.Categories (Union)

4. **Aggregation:**
   - Sometimes we have to model a relationship between a collection of entities and relationships.
   - Basic ER model cannot express relationships among relationship sets and relationships between relationship sets and entity sets.
   - It is a process that represent a relationship between a whole object and its component parts.
   - It shows a relationship between objects and viewing the relationship as an object.
   - It is a process when two entity is treated as a single entity.

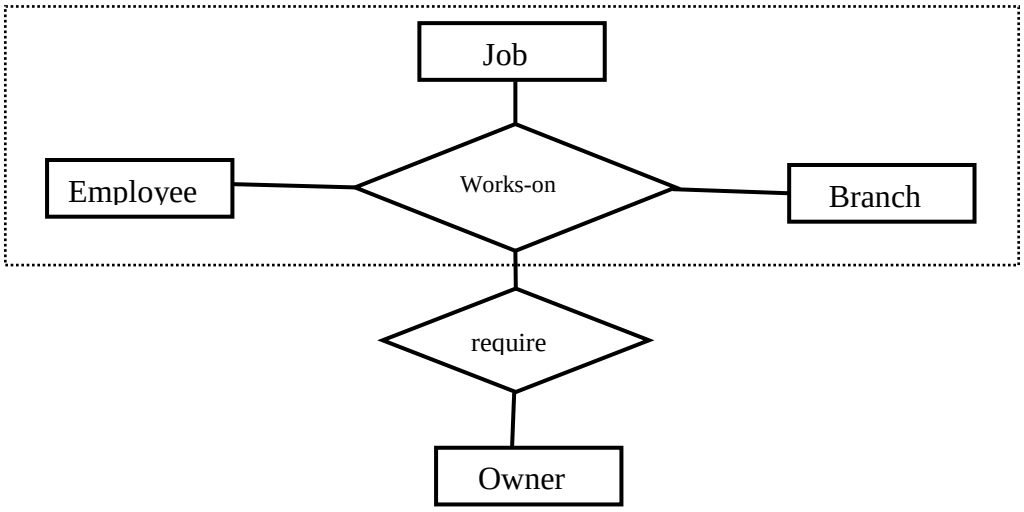


Figure 13 Aggregation

Constraints on Specialization and Generalization:

- To model real world more accurately by using ER diagram we need to keep certain constraints on it.
- Constraints on which entities can be members of a given lower-level entity set are discussed below:
  - Participation constraints
  - Disjoint vs. Overlap Constraints
  - A total specialization vs. a partial specialization

Participation constraints:

- Participation constraint specifies whether every entity in a superclass must also be a member of at least one subclass.
- It determines the extent of participation of entities in the superclass within the subclass relationship.
- It can be of two types:
  - **Mandatory (Total) Participation:** Every entity in the superclass must belong to at least one subclass.
  - **Optional (Partial) Participation**: An entity in the superclass may or may not belong to any subclass.

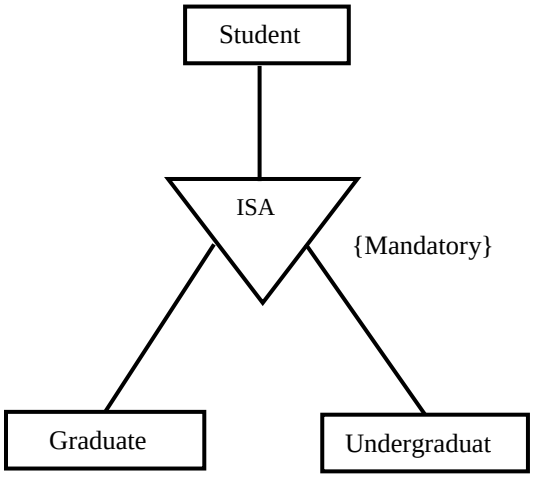


Figure 15 Mandatory participation

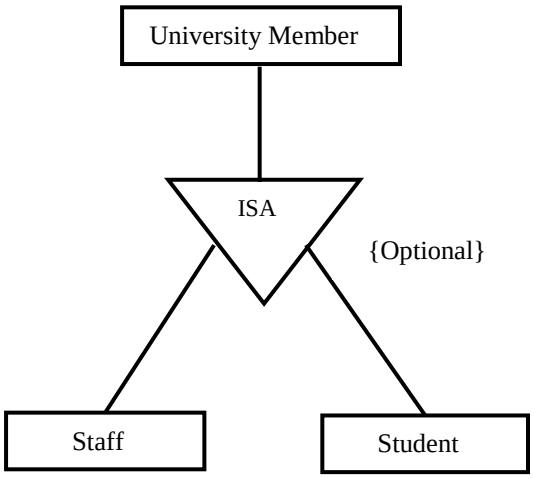


Figure 14 Optional Participation

Disjoint vs. Overlap Constraints

- If an entity can be a member of at most one of the subclasses of the specialization, then the subclasses are called disjoint.
- In EER diagram, d in the circle stands for disjoint. For example, specialized subclasses 'Electronic Book' and 'Paperback Book' are disjoint subclasses of the superclass entity set Book.
- If the same entity may be a member of more than one subclass of the specialization, then the subclasses are said to be overlapped.
- For example, specialized subclasses 'Action Movie' and 'Comedy Movie' are overlapped subclasses of the superclass entity set Movie.
- This is because there also movies called "Action Comedy" that belong both of the above subclasses.

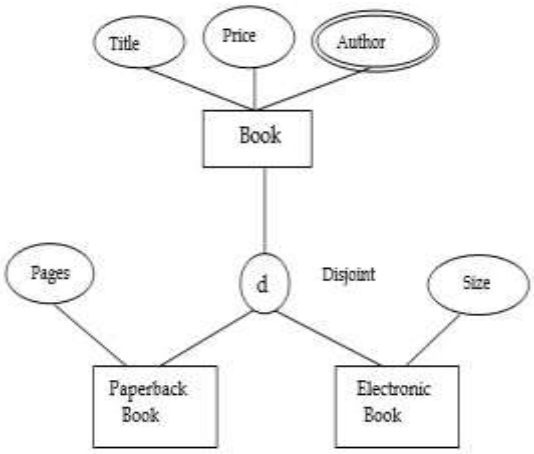


Figure 16 Disjoint constraint

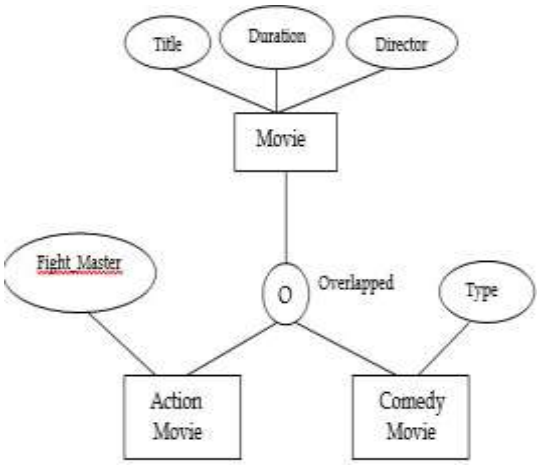


Figure 17 Overlapped constraint

A total specialization vs. a partial specialization:

- A total specialization constraint specifies that every entity in the superclass must be a member of some subclass in the specialization.
- It is represented by a double line connecting the superclass to the circle.
- A partial specialization allows an entity not to belong to any of the subclasses, using a single line in EER. e.g., if some EMPLOYEE entities, for example, **sales representatives**, do not

belong to any of the subclasses {SECRETARY, ENGINEER, TECHNICIAN}, then the specialization is partial.

- It is represented by a single line connecting the superclass to the circle.
- Partial generalization is the default.

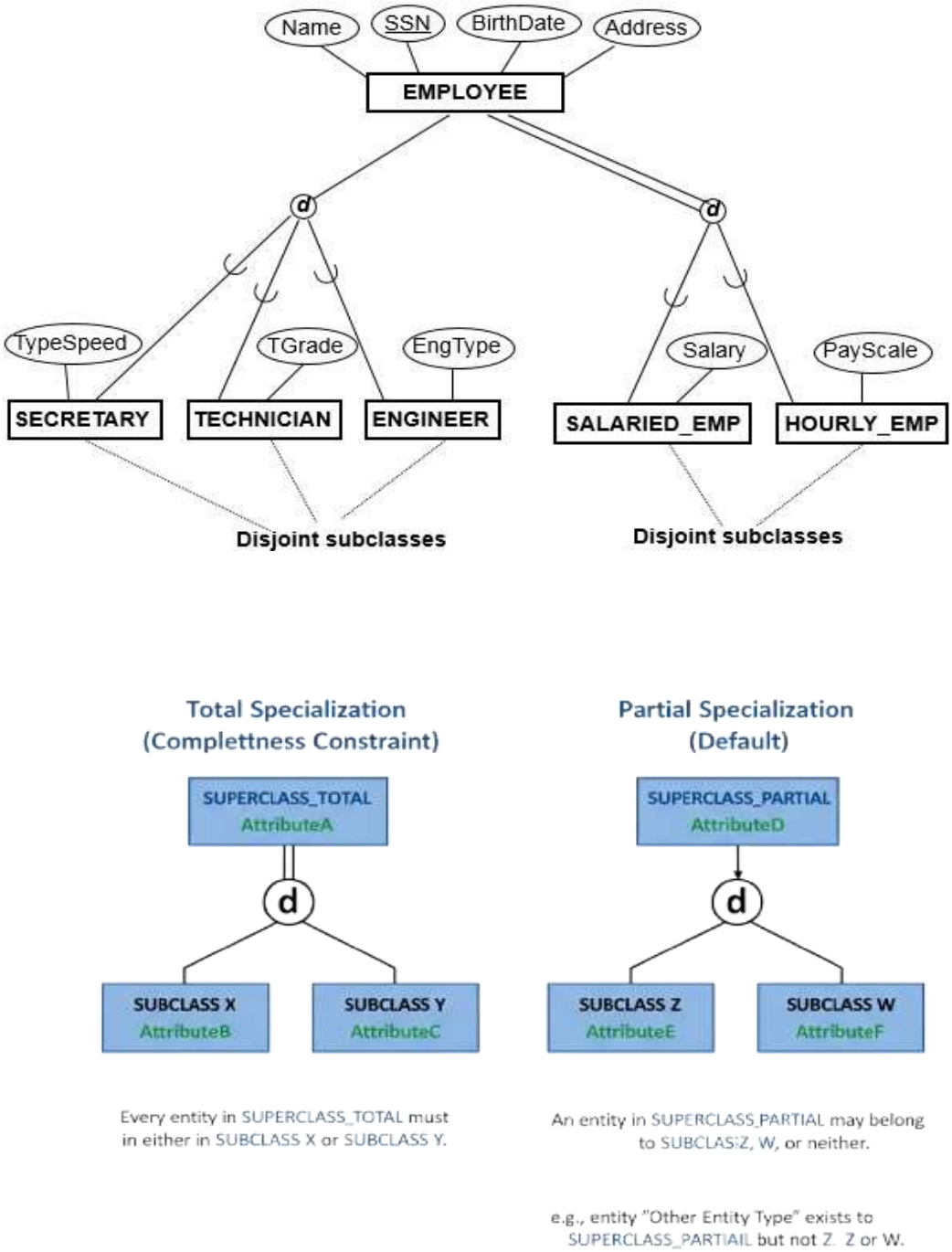


Figure 18 Total vs partial specialization

Types of Attributes:

- Atomic vs composite attributes
    - An attribute that cannot be divided into smaller independent attributes is known as an atomic attribute.
    - Assume "Student" is an entity and its attributes are {Name, Age, DoB, Address, Phon No.}. Here, Std_id and DoB of Student (entity) cannot further divide. So that Std_id and DoB are atomic attributes.
    - An attribute that can be divided into smaller independent attributes is known as composite attributes.
    - Assume "Student: is an entity and its attributes are {Name, Age, DoB, Address, Phon No.}. Here, the Address (attribute) of Student (entity) is further divided into House No., City so on. So, Address is a composite attribute.
- Single-valued vs multi-valued attributes
    - An attribute that has only single value for an entity is known as single-valued attribute.
    - Assume "Student: is an entity and its attributes are {Name, Age, DoB, Address, Phon No.}. Here, the Age (attribute) of Student (entity) can have only one value. Here, Age is single-valued attribute.
    - An attribute that can have multiple values for an entity is known as multi-valued attribute.
    - Assume "Student: is an entity and its attributes are {Name, Age, DoB, Address, Phon No.}. Here, the Phone No. (attribute) of Student (entity) can have multiple values because a student may have many Phone No. Here, Phone No. is multi-valued attribute.
- Stored vs derived attributes
    - An attribute that cannot be derived from another attribute and we need to store its value in the database is known as a stored attribute.
    - Fore example, birth date cannot be derived from age of student.
    - An attribute that can be derived from another attribute and we do not need to store their value in the database due to dynamic nature is known as derives attribute.
    - It is denoted by oval.

- NULL value attribute
  - Attributes that do not have any value for an entity is known as null-valued attributes.
  - They are also called unknown and missing values.
  - For example, if a social security value of a particular customer is null, we assume, the value is missing.
- Key attribute
  - An attribute that has unique value of each entity is known as key attribute.
  - For example, every student has a unique Stu_id. Here, Stu_id is key attribute.

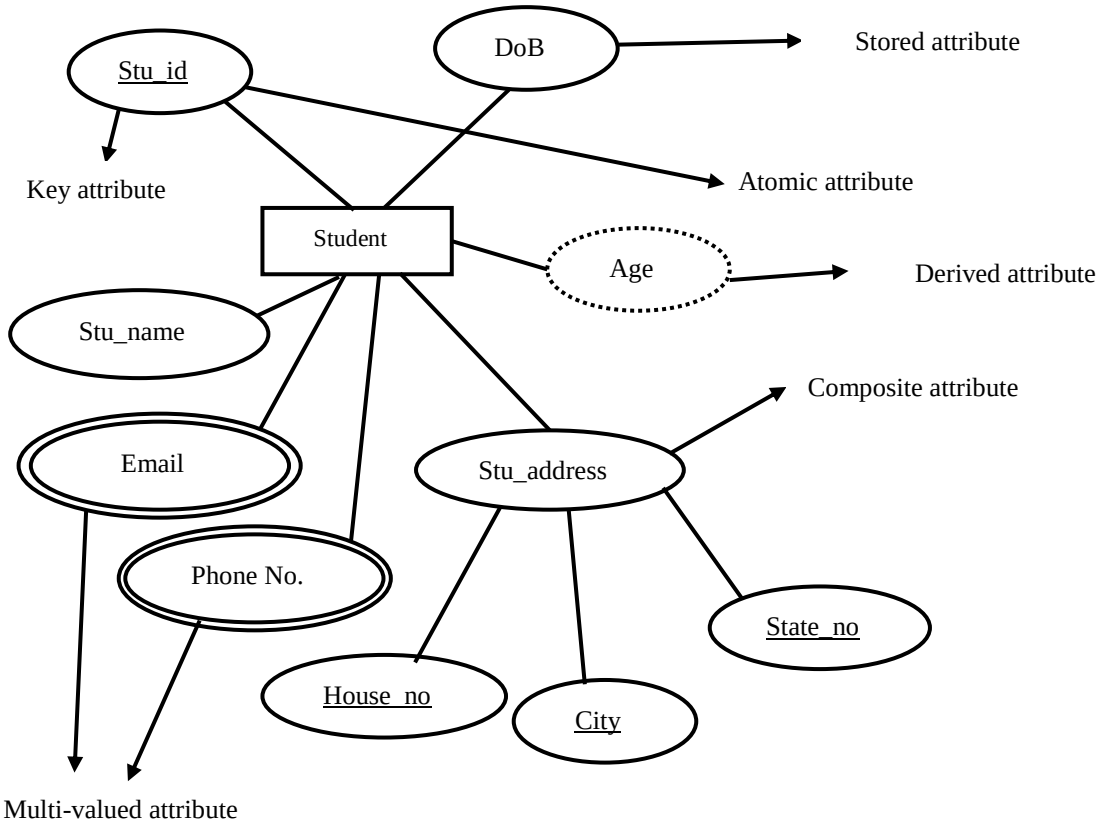


Figure 19 Types of Attributes

Degree of Relationship:

Number of entity sets that participate in a relationship set is called the degree of the relationship set. Based on degree, relationships can be divided as below.

- **Unary Relationship**
  - A relationship where only one entity set participates.
  - The entity set is related to itself.
  - Example: an employee supervises another employee.

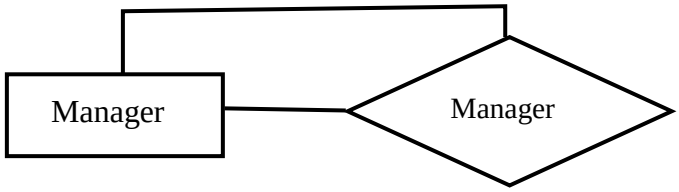


Figure 20 Unary Relationship

- **Binary Relationship**
  - A relationship involving two different entity sets.
  - It is the most common type of relationship in ER models.
  - Example: Student – Enrolled in – Course.

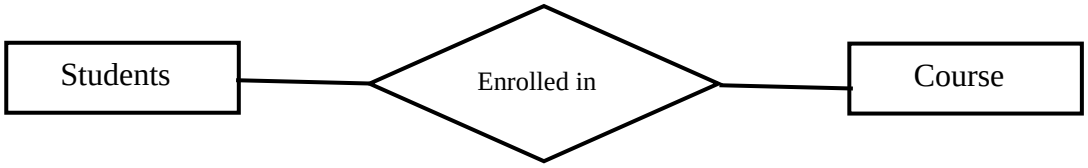


Figure 21 Binary relationship

- **Ternary Relationship**
  - A relationship involving three entity sets at the same time.
  - It cannot be broken down into three binary relationships without losing meaning.
  - Example: Employee – Works in – Department – Location

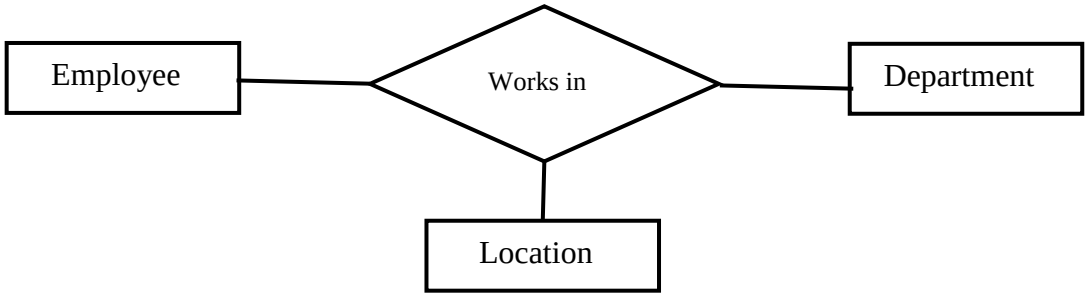


Figure 22 Ternary relationship

- **N-ary Relationship**
  - A relationship involving N entity set, where $N \geq 2$
  - Binary and ternary are special cases of n-ary.
  - Used when multiple entities must participate together to define a relationship.

Mapping Cardinality Constraints

- Specifies the maximum number of relationship instances in which an entity can participate.
- Also known as the cardinality ratio.
- Used in the ER model to define how entities relate to each other.

1. One-to-One (1:1) Relationship

- Occurs when each entity in set A is related to at most one entity in set B, and each entity in set B is also related to at most one entity in set A.
- Represents a unique-to-unique association.
- Example:
  - A bank has only one CEO, and a person can be the CEO of only one bank.
  - Therefore, the relationship between Bank and CEO is one-to-one.

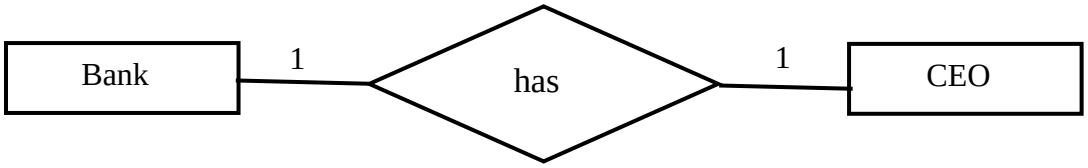


Figure 23: One to One Relationship

2. One-to-Many (1:N) Relationship

- Occurs when one entity in set A can be related to many entities in set B, but each entity in set B can be related to only one entity in set A.
- Represents a single-to-multiple association.
- Example:
  - A mother can have multiple children, but each child has only one mother.
  - Thus, the relationship between Mother and Child is one-to-many.

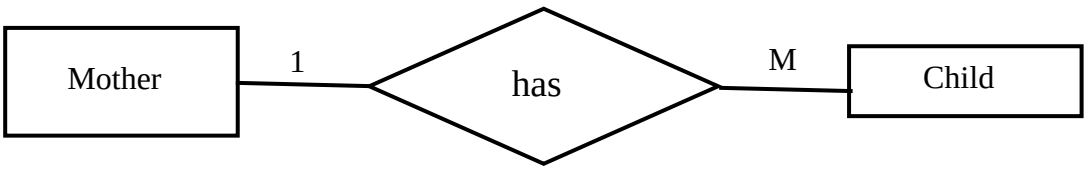


Figure: 24 One to Many Relationship

3. Many-to-One (N:1) Relationship

- Occurs when many entities in set A can be related to one entity in set B, but each entity in set A relates to only one entity in set B.
- Represents a multiple-to-single association.
- Example:
    - A book is published by one publisher, but a publisher can publish many books.
    - Therefore, the relationship between Book and Publisher is many-to-one.

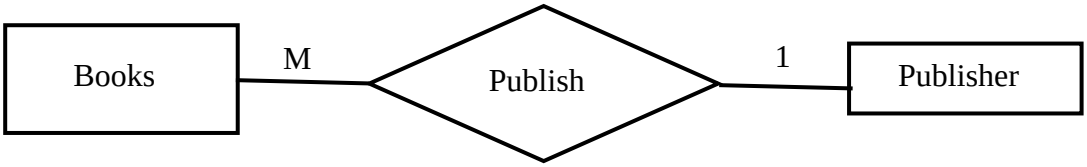


Figure 25: Many to One Relationship

4. Many-to-Many (N:N) Relationship

- Occurs when entities in A can be related to many entities in B, and entities in B can also be related to many entities in A.
- Represents a multiple-to-multiple association.
- Example:
    - A student can enroll in multiple subjects, and a subject can be taken by many students.
    - So, the relationship between Students and Courses is many-to-many.

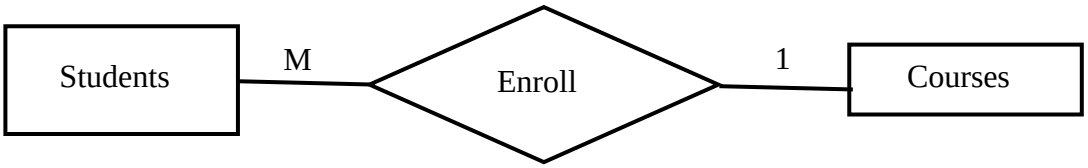


Figure 26: Many to Many Relationship

Relational Model

- The relational data model represents the logical view of how data is stored in the relational database.
- In the relational model, the data and relationships are represented by a collection of interrelated tables.

- Each table is a group of columns and rows, where column represents attribute of an entity and rows represent records.

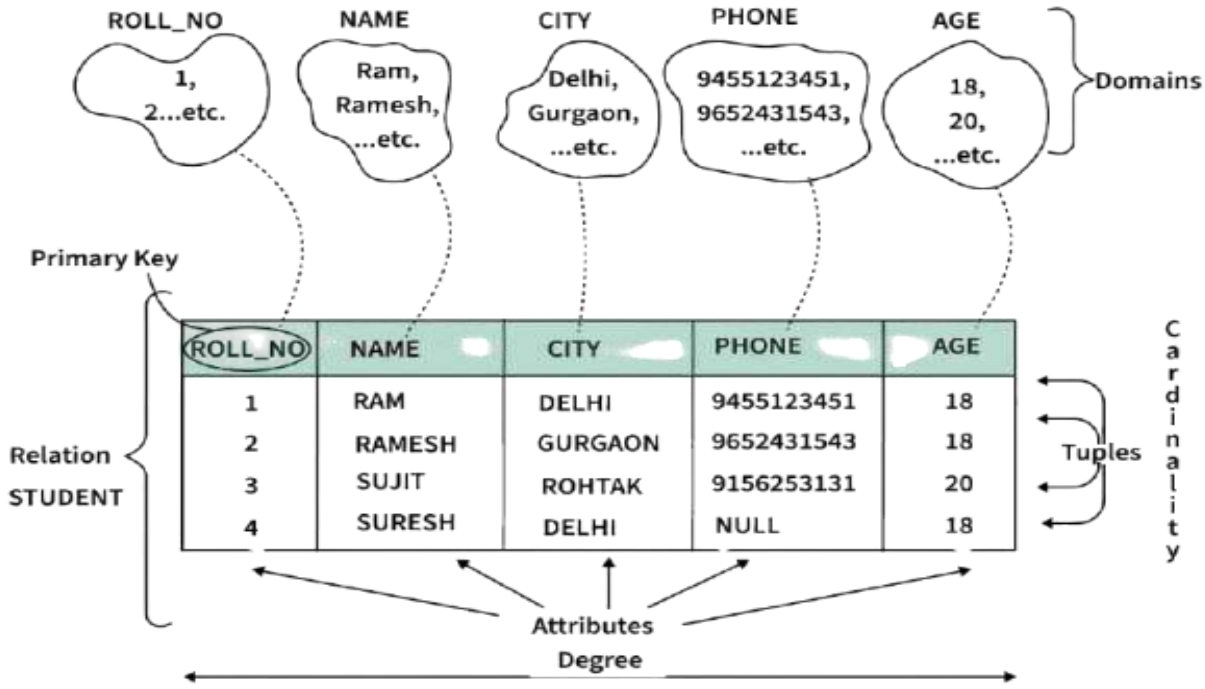


Figure 27 Relational Model

Strong Entity in DBMS

- A Strong Entity is independent of any other entity in the schema.
- Example: A student entity can exist without needing any other entity in the schema or a course entity can exist without needing any other entity in the schema.
- A Strong entity is nothing but an entity set having a primary key attribute or a table that consists of a primary key column.
- The strong entity is represented by a single rectangle.
- The relationship between two strong entities is represented by a single diamond.
- Example: Consider the ER diagram, which consists of two entities: student and course and here, student entity is a strong entity because it consists of a primary key called student id which is enough for accessing each record uniquely.

- In the same way, the course entity contains of a course ID attribute which is capable of uniquely accessing each row.

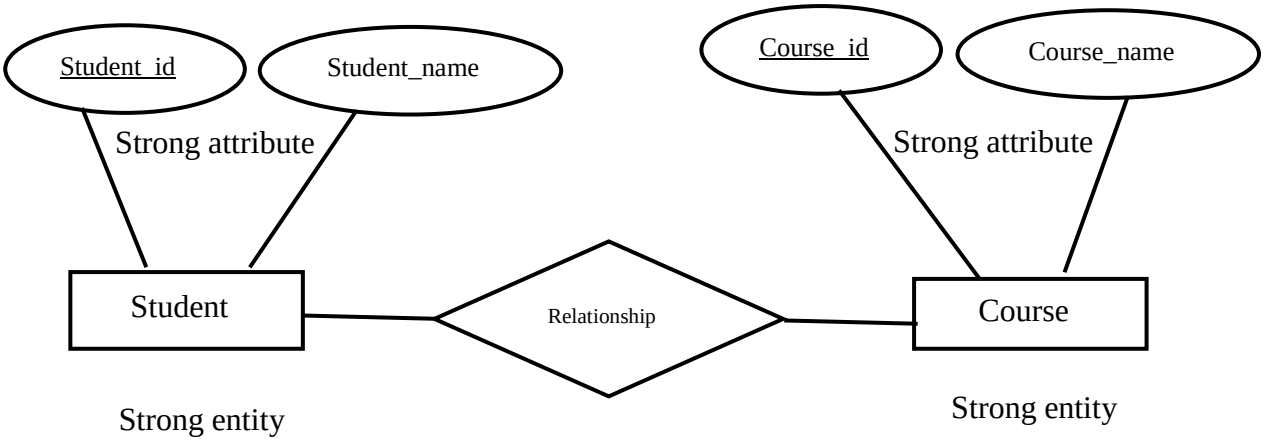


Figure 28: Strong entity and strong attribute

Weak Entity in DBMS

- A weak entity is an entity set that does not have sufficient attributes for the Unique Identification of its records.
- Example: A loan entity can not be created for a customer if the customer doesn't exist
- Also, A dependents list entity can not be created if the employee doesn't exist
- Simply, a weak entity is nothing but an entity that does not have a primary key attribute
- It contains a partial key called a discriminator which helps in identifying a group of entities from the entity set
- A discriminator is represented by underlining with a dashed line.
- A double rectangle is used to represent a weak entity set.
- The double diamond symbol is used to represent the relationship between a strong entity and a weak entity, which is known as an identifying relationship.

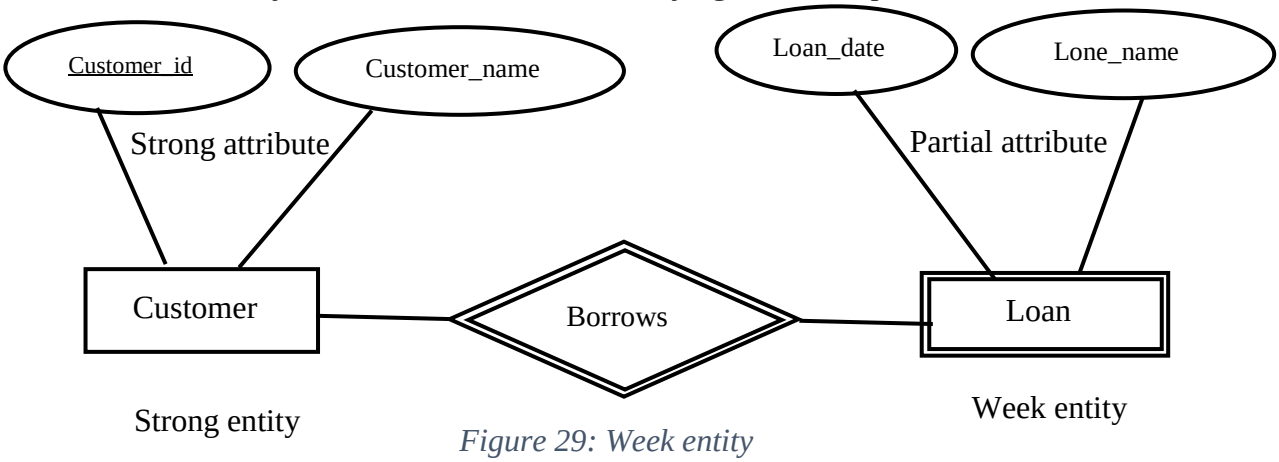


Figure 29: Week entity

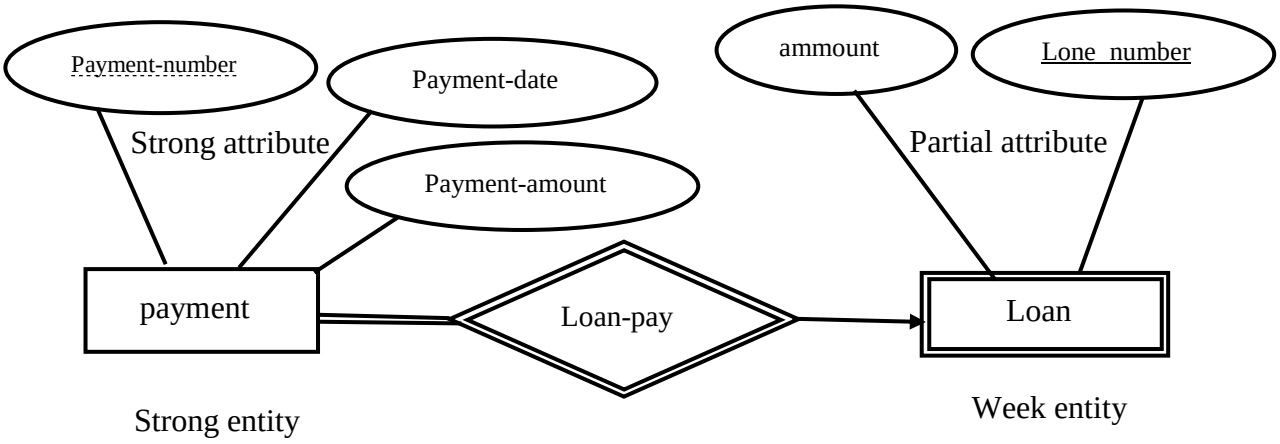


Figure 30: Strong and week entity

Join Operation:

Assignment-1: What is join in DBMS? What are the join operations? Explain each type with examples.

Converting ER and EER Model to Relational Model:

- ER and EER diagrams can be transformed into a relational schema.
- Not all constraints from ER/EER can be directly converted, but an approximate relational model can be created.
- After designing an ER/EER diagram, we convert it into a relational model so it can be implemented in databases like Oracle, MySQL, etc.
- The chapter discusses how to convert ER and EER diagrams to relational models in different scenarios.
- The ER to relational mapping algorithm consists of the following steps:
  - Step 1: Mapping of Regular Entity Types
  - Step 2: Mapping of Weak Entity Types
  - Step 3: Mapping of Multivalued attributes.
  - Step 4: Mapping of Composite attributes.
  - Step 5: Mapping of Binary 1:1 Relation Types.
  - Step 6: Mapping of Binary 1: N Relationship Types.
  - Step 7: Mapping of Binary M: N Relationship Types.

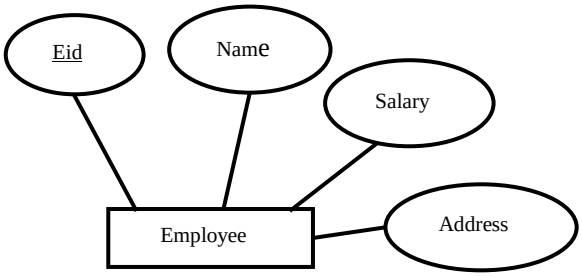
- Step 8: Mapping of N-ary Relationship Types.

Then, the EER to relational mapping algorithm consists of the following extra steps:

- Step 9: Mapping Specialization/Generalization to ER.
- Step 10: Mapping Aggregation to ER.

Step-1: Mapping regular entity set to relational model

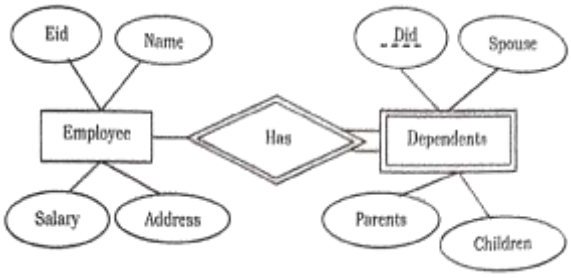
- An entity set is mapped to a relation in a straightforward way:
- Each simple attribute of the entity set becomes an attribute (column) of the table and primary key of the entity set becomes primary key of the relation.
- Domain of each attribute of the relation must be known.



Eid	Name	Salary	Address

Step-2: Mapping of Weak Entity Types

- A weak entity set does not have its own primary key and always participates in one-to-many relationship with owner entity set and has total participation.
- For a weak entity set create a relation that contains all simple attributes (or simple components of composite attributes).
- In addition, relation for weak entity set contains primary key of the owner entity set as foreign key and its primary key is formed by combining partial key (discriminator) and primary key of the owner entity set.



Employee

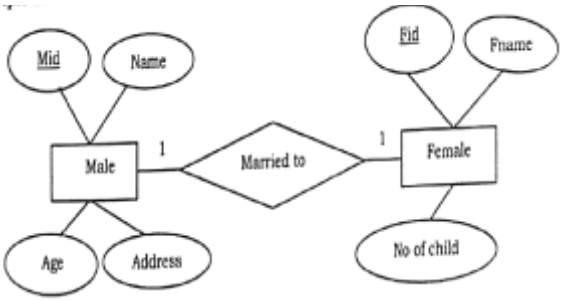
Eid	Name	Salary	Address

Dependents

Did	Did	Parents	Spouse	Children

Step-3: Mapping of Multivalued and composite attributes

- If an entity has composite attributes, no separate attribute (column) is created for composite attribute itself rather attributes (columns) are created for component attributes of the composite attribute.
- For a multivalued attribute, separate relation is created with primary key of the entity set and multivalued attribute itself.



Male

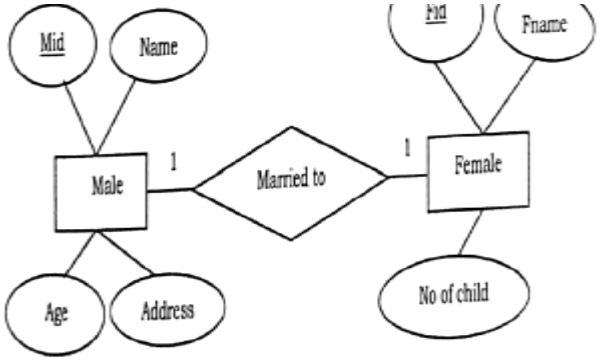
Mid	Name	Age	Address

Female

Fid	Mid	Fname	No of child

Step-4: Mapping of Binary 1:1 Relation Types

- For a binary one to one relationship set separate relation is not created rather primary key of an entity set is included in relation for another entity set as a foreign key.
- It is better to include foreign key into the entity set with total participation in the relationship.



Male

Mid	Name	Age	Address

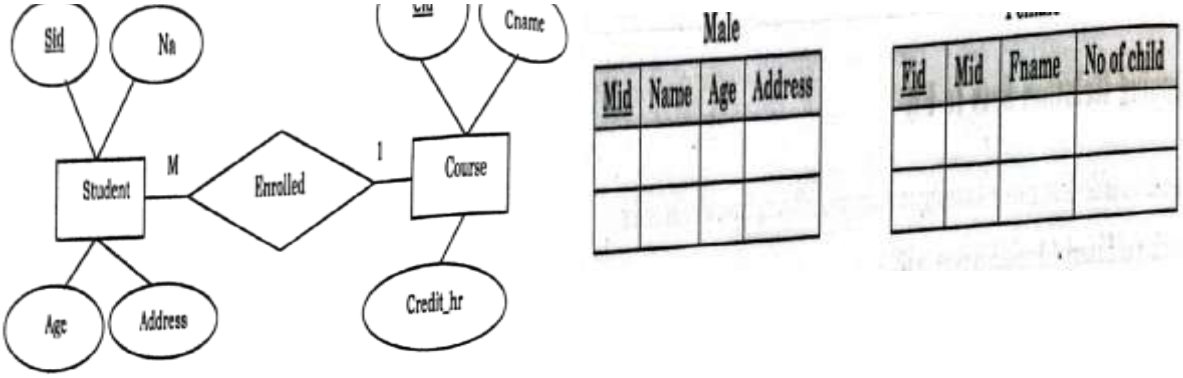
Female

Fid	Mid	Fname	No of child

Step-5: Mapping of Binary 1: N Relation Types

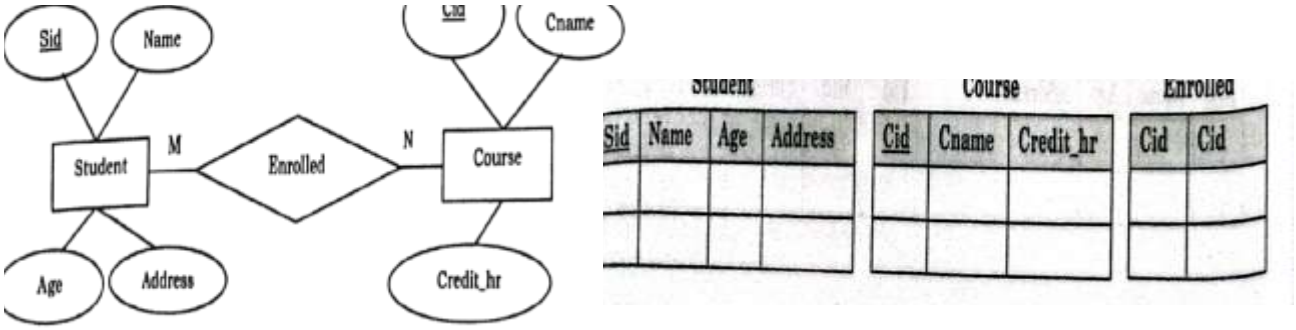
- For binary one-to-many relationship identify the relation that represent the participating entity type at the N-side of the relationship type and then include primary key of one side entity set into many side entity set as foreign key.

- Separate relation is created for the relationship set only when the relationship set has its own attributes



Step-6: Mapping of Binary M: N Relation Types

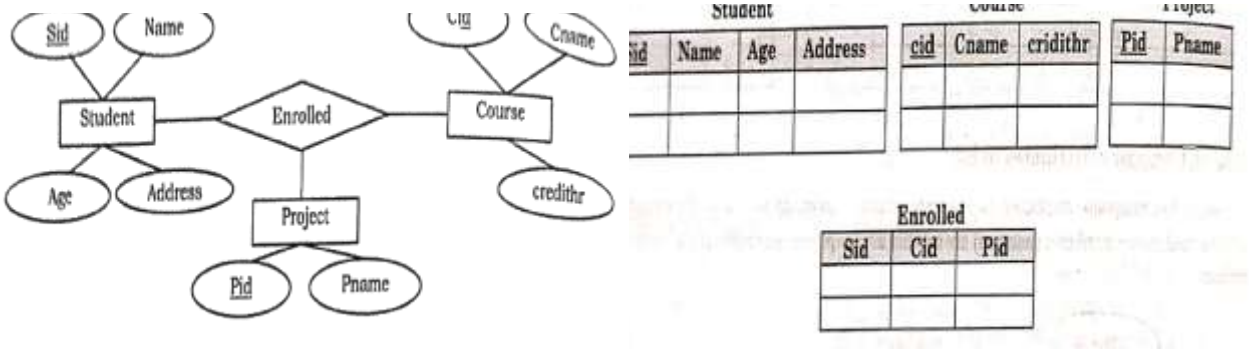
- For a binary Many-to-Many relationship type, separate relation is created for the relationship type.
-
- Primary key for each participating entity set is included as foreign key in the relation and their combination will form the primary key of the relation.
- Besides this, simple attributes of the many-to-many relationship type (or simple components of composite attributes) is included as attributes of the relation.



Step-7: Mapping of N-ary Relationship Types

- For each n-ary relationship set for n>2, a new relation is created. Primary keys of all participating entity sets are included in the relation as foreign key attributes.

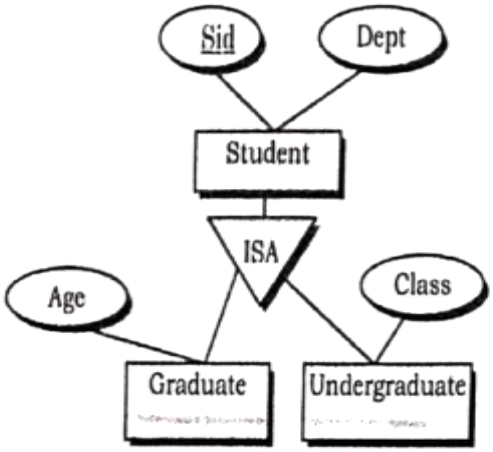
- Besides this all simple attributes of the n-ary relationship set (or simple components of composite attributes) are included as attributes of the relation.

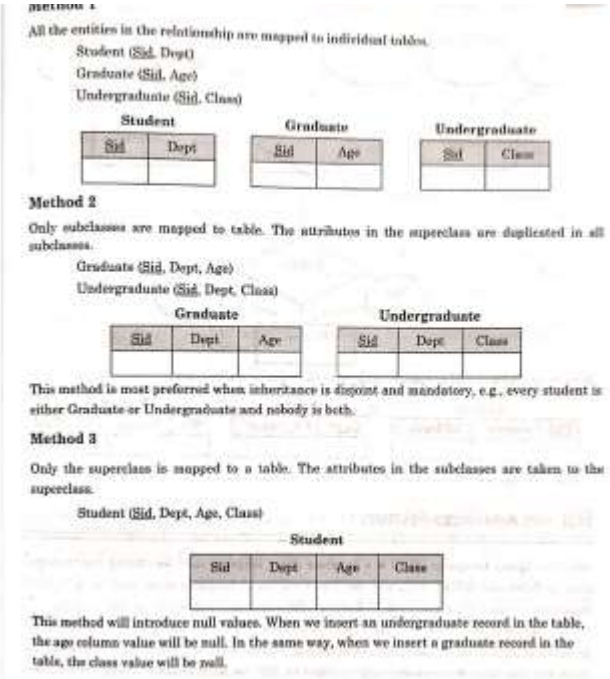


Step-8: Representing Specialization/Generalization

- There are two approaches for translating ER diagrams with specialization or generalization into relations.

1. A relation is created for higher level entity set using above discussed methods and then separate relation is created for each of the lower level entity sets. Relation for a subclass entity set includes all of its attributes and key attributes of superclass entity set.
2. Another approach is to create relations only for lower level entity sets. Here, relation for a subclass entity set includes all attributes of superclass entity set and all of its own attributes. This approach is possible only when subclasses are disjoint and complete




All the entities in the relationship are mapped to individual tables.

Student (Sid, Dept)

Graduate (Sid, Age)

Undergraduate (Sid, Class)

Student | Sid | Dept |

Graduate | Sid | Age |

Undergraduate | Sid | Class |

Method 2

Only subclasses are mapped to table. The attributes in the superclass are duplicated in all subclasses.

Graduate (Sid, Dept, Age)

Undergraduate (Sid, Dept, Class)

Graduate | Sid | Dept | Age |

Undergraduate | Sid | Dept | Class |

This method is most preferred when inheritance is disjoint and mandatory, e.g., every student is either Graduate or Undergraduate and nobody is both.

Method 3

Only the superclass is mapped to a table. The attributes in the subclasses are taken to the superclass.

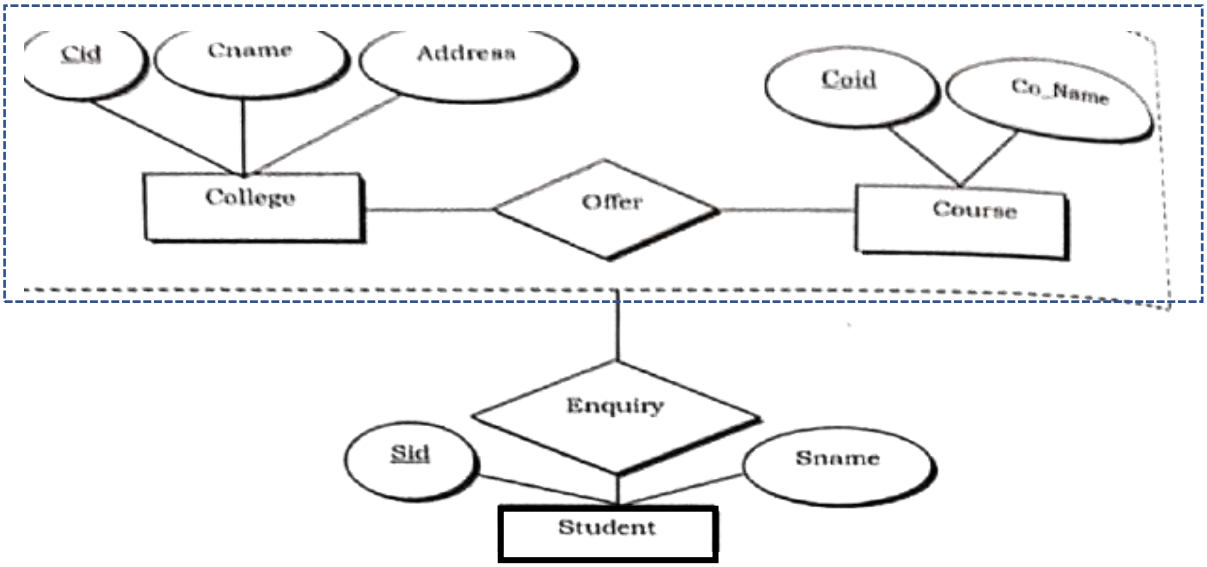
Student (Sid, Dept, Age, Class)

Student | Sid | Dept | Age | Class |

This method will introduce null values. When we insert an undergraduate record in the table, the age column value will be null. In the same way, when we insert a graduate record in the table, the class value will be null.

Step-9: Representing Aggregation

- To represent aggregation, create a table containing
  - ❖ Primary key of aggregated relationship
  - ❖ Primary key of the associated entity set
  - ❖ Any descriptive attributes of the relationship set



College

Cid	cname	Address

Course

Coid	Co_Name

Enquiry

Sid	Sname	Cid	Coid

SQL and Advanced Features:

- SQL stands for Structured Query Language.
- It is a database query language used for storing and managing data in relational databases and tables.
- It is a standard language for relational database systems.
- It enables a user to create, read, update, and delete relational databases and tables.
- SQL was developed in the early 1970s at IBM by Donald D. Chamberlin and Raymond F. Boyce. It was originally called SEQUEL (Structured English Query Language) before being renamed SQL.
- SQL is the standard language for relational database systems, adopted by ANSI (American National Standards Institute) and ISO.
- It enables a user to create, read, update, and delete relational databases and tables (also known as CRUD operations).
- SQL supports Data Definition Language (DDL), Data Manipulation Language (DML), Data Control Language (DCL), and Transaction Control Language (TCL).
- SQL supports constraints (Primary Key, Foreign Key, Unique, Check, Not Null) to maintain data integrity.
- SQL offers joins (INNER, LEFT, RIGHT, FULL) to combine records from multiple tables.
- SQL supports advanced features like views, stored procedures, functions, triggers, and indexes to improve efficiency and modularity.
- SQL is used in popular database systems such as MySQL, PostgreSQL, Oracle, SQL Server, and SQLite.

SQL Features:

1. Data Definition Language (DDL):

- SQL includes commands for defining and managing the structure of a database.
- Key DDL commands include:
    - CREATE: Used to create database objects such as tables, indexes, and views.

      CREATE TABLE Student(RollNumber varchar(20), StudentName varchar(255), CollegeName varchar(255), Address varchar(255));
    - ALTER: Used to modify the structure of existing database objects.

      ALTER TABLE Student ADD Email varchar(255);
    - DROP: Used to delete database objects.

      DROP TABLE Student;
    - TRUNCATE: Used to remove all records from a table, including all spaces allocated for the records are removed.

      TRUNCATE TABLE Student;

- COMMENT: Used to add comments to the data dictionary. Use "–" or this "/* */" symbol for comment.

  TRUNCATE TABLE Student;

  or

  / TRUNCATE TABLE Student; */*
- RENAME: Used to rename an object existing in the database.

  ALTER TABLE Student RENAME TO ITstudent;

Data Manipulation Language (DML):

- SQL supports commands for manipulating data stored in the database.
- Key DML commands include:
  - SELECT: Used to retrieve data from the database.

    *SELECT * FROM Student;*
  - INSERT: Adds new records to a table.

    INSERT INTO Student(RollNumber, StudentName varchar, CollegeName, Address)VALUES ('123CS2019', 'Jayesh', 'ATC', 'Indore');
  - UPDATE: Modifies existing records in a table.

    UPDATE Student SET StudentName = 'Kumar', City= 'Burhanpur'

    WHERE RollNumber = '123CS2019';
  - DELETE: Removes records from a table.

    DELETE FROM Student WHERE RollNumber='123CS2019';

Data Control Language (DCL):

- DCL commands are used to manage access to data within the database.
- Key DCL commands include:
  - GRANT: Provides specific privileges to users or roles.

    GRANT SELECT, INSERT ON Student TO 'Jayesh';
  - REVOKE: Removes specific privileges from users or roles.

    REVOKE SELECT, INSERT ON Student TO 'Jayesh';

Transaction Control Language (TCL):

- SQL supports transactions, which are sequences of one or more SQL statements that are executed as a single unit.

- Transactions ensure data consistency and integrity.
- Key transaction control commands include:
    - COMMIT: Saves all the changes you made during a transaction permanently

```
BEGIN;
- - SQL statements
COMMIT;
```

    - ROLLBACK: Undoes changes you made during a transaction if something went wrong.

```
BEGIN;
- - SQL statements
ROLLBACK;
```

    - SAVEPOINT: Sets a point inside a transaction to which you can roll back if needed.

```
BEGIN;
- - SQL Statements
SAVEPOINT My_savepoint;
- - More SQL statements
ROLLBACK TO my_savepoint;
```

Advanced Features of AQL:

SQL Union:

- Combining and deduplicating data.
- The UNION operation merges the result of two SELECT statements, eliminating duplicates.

  Two input tables "employee1" and "employee2" are:

employee1

ID	Name
1	Alice
2	Bob
3	Charlie
4	David
5	Eve

employee2

ID	Name
3	Charlie
4	David
5	Eve
6	Frank
7	Grace

Query:

```
SELECT ID, Name FROM employee1
UNION
SELECT ID, Name FROM employee2;
```

Output Result from UNION:

ID	Name
1	Alice
2	Bob
3	Charlie
4	David
5	Eve
6	Frank
7	Grace

SQL Union All

- Combining data without deduplication.
- UNION ALL combine result without removing duplicate providing a faster performance compared to UNION.

Query:

```
SELECT ID, Name FROM employee1
UNION ALL
SELECT ID, Name FROM employee2;
```

Output Result from UNION ALL:

ID	Name
1	Alice
2	Bob
3	Charlie
4	David
5	Eve
3	Charlie
4	David
5	Eve
6	Frank
7	Grace

SQL Intersect

- Finding common entries.
- INTERSECTION returns rows common to both queries, finding the intersection of two result sets.

Query:

```
SELECT ID, Name FROM employee1
INTERSECT
SELECT ID, Name FROM employee2;
```

Output Result from INTERSECT:

ID	Name
3	Charlie
4	David
5	Eve

SQL Minus

- Finding differences.
- MINUS return unique rows from the first query that do not exist in the second.

Query:

```
SELECT ID, Name FROM employee1
MINUS
SELECT ID, Name FROM employee2;
```

Output Result from MINUS:

ID	Name
1	Alice
2	Bob

SQL Limit

- LIMIT tells the databases how many rows (records) you want to see.
- Set OFFSET define functionality.

Query:

```
SELECT ID, Name FROM employee1
LIMIT 3;
```

Output Result from LIMIT:

ID	Name
1	Alice
2	Bob
3	Charlie

Concepts of File Structures:

- Relative data and information is stored collectively in file formats.
- A file is a sequence of records stored in binary format.
- A disk drive is formatted into several blocks that can store records.
- Database is stored in a collection of files.
- Each file is a sequence of records.
- A record is a sequence of fields.
- File Organization defines how file records are mapped onto disk blocks.
- If every record in the file has exactly the same size (in bytes), the file is said to be made up of fixed-length records.
- If different records in the file have different sizes, the file is said to be made up of variable-length records.
- We have four types of File Organization to organize file records:
    1. **Sequential file organization:**
        - Every file record contains a data field (attribute) to uniquely identify that record.
        - In sequential file organization, records are placed in the file in some sequential order based on the unique key field or search key.
        - Practically, it is not possible to store all the records sequentially in physical form.
        -

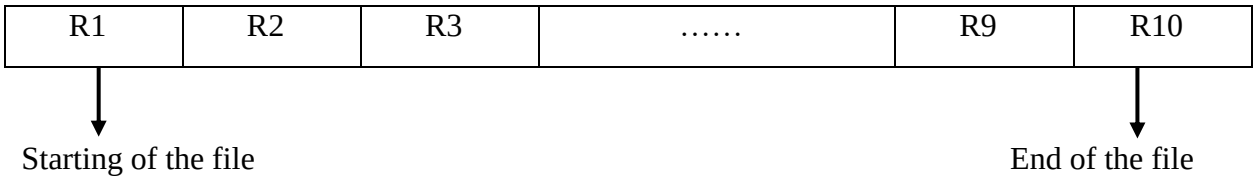


Figure 31: Sequential file structure

2. **Heap file organization:**
    - Heap file organization is the simplest method of storing records in a file.
    - Records are inserted in the order they arrive, usually at the end of the file.
    - This makes insertion very fast, because the system just adds the new record to the last block.
    - When deleting a record, the system must locate the record, load its block into memory, remove it, and then write the block back to disk.
    - The operating system only provides a memory area for the file; it does not maintain any structure.
    - Records can be stored anywhere within this memory area, with no fixed order.
    - There is no built-in sorting, sequencing, or indexing in a heap file. The DBMS itself must manage how records are accessed.
    - In simple words, *"a record can be placed anywhere in the file where there is space"*.
    - It has data insert operation fast but deletion and search operation is slow.

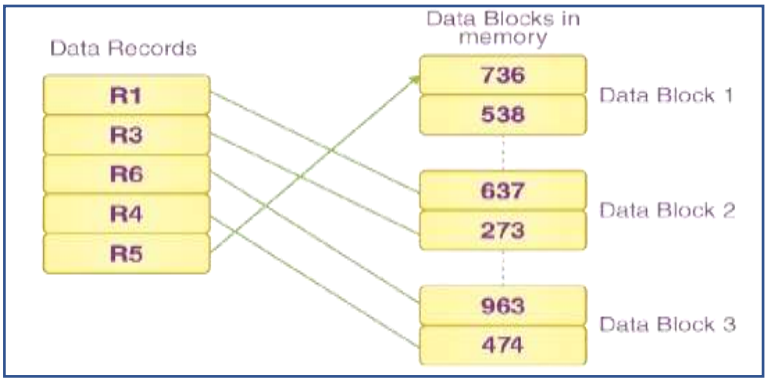


Figure 32: Heap file structure

3. **Hash file organization:**
    - Hash file organization stores records based on the result of a hash function.
    - A hash function takes a selected field of a record (called the hash key) and computes a value.
    - This computed value determines the exact disk block where the record will be stored.
    - Because each record is mapped directly to a block using the hash value, searching becomes very fast.

- In simple terms, the hash function acts like a formula that tells the DBMS where a record should be placed on the disk.
- **Bucket:** In a hash file, data is stored in buckets, which act as the basic units of storage. A bucket usually corresponds to one disk block, and each block can hold one or more records.
- **Hash Function:** A hash function (h) is a mapping function that converts a search key (K) into a bucket address. It determines the exact location where the corresponding record will be stored in the hash file.

$$H(K) = K \bmod M$$

Static Hashing:

- In static hashing, the hash function always produces the same bucket address for a given search key.
- Example: A mod-4 hash function always produces one of four fixed addresses.
- The number of buckets remains constant and does not change even if the amount of data grows.
- Here are some static hashing operations:

Insertion:

- Apply the hash function to the key to get a bucket index.
- Place the record in the corresponding bucket.
- If the bucket already contains records, handle the collision using the chosen method (e.g., chaining).

Search:

- Apply the hash function to the key to get the bucket index.
- Search within the bucket for the desired record.
- This search can be linear or use more sophisticated structures like binary search if the bucket is kept sorted.

Deletion:

- Apply the hash function to find the bucket.
- Search for the record within the bucket and remove it.
- Adjust any linked list or probing structures accordingly.

Bucket Overflow (Collision) occurs when multiple keys map to the same bucket and the bucket becomes full. Bucket overflow is a common issue in static hashing.

Collision Handling Techniques:

Overflow Chaining (Closed Hashing):

- When a bucket becomes full, a new bucket is allocated and linked to the original bucket.

Linear Probing (Open Hashing):

- If the computed bucket is full, the system stores the record in the next available bucket.
- For static hashing to work efficiently:
- Record distribution should be uniform.
- The hash output should be random, not ordered.

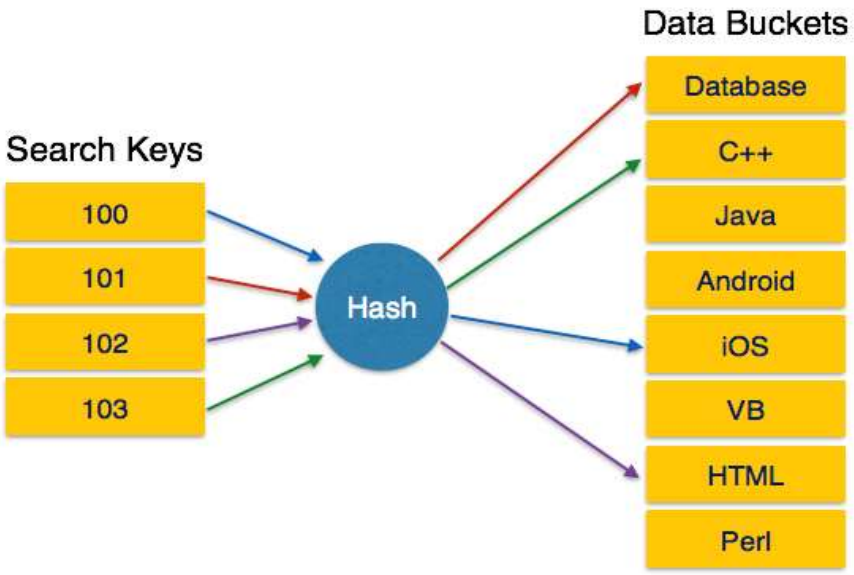


Figure 33: Static Hashing

Dynamic Hashing:

- Static hashing cannot handle changes in database size, so dynamic hashing allows the structure to expand or shrink automatically.
- It adds or removes data buckets dynamically, based on the number of records.
- Dynamic hashing is also known as extended hashing.
- The hash function is designed to produce many possible values, but only a subset is used initially. As the database grows:
- More bucket addresses are activated.
- New buckets are allocated to avoid overflow.

- As the database shrinks:
- Unused buckets may be released.
- The structure becomes smaller and more efficient.
- Dynamic hashing minimizes collisions because the system adapts the number of buckets according to data size.

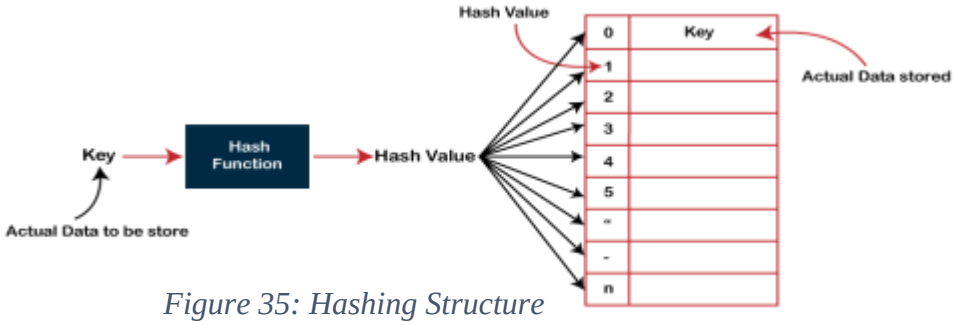


Figure 35: Hashing Structure

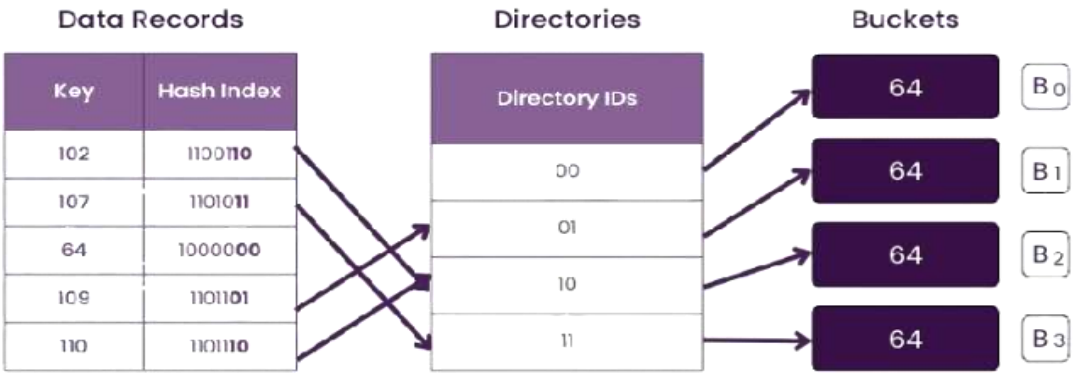


Figure 34: Dynamic Hashing

Operations in Hashing:

- **Search:** Check the hash index, use the required number of bits, and compute the bucket address to find the data.
- **Update:** First perform a search to locate the record, then update it.
- **Deletion:** Search for the record using the hash value, then delete it from the bucket.
- **Insertion:** Calculate the bucket address using the hash function.
    - If the bucket is full:
        - Allocate extra buckets, or
        - Increase the number of hash bits to create more addresses.

4. B+ Tree file organization:

- It is alternative to index sequential files.

- B-Tree is one of the most commonly used tree-based indexing structures in DBMS.
- It is a multi-level, balanced tree, meaning all paths from the root to any leaf have roughly the same length.
- Leaf nodes contain the actual data pointers, i.e., they point to the real records.
- All leaf nodes are connected using a linked list, so the B-Tree supports both random access (jumping directly to data) and sequential access (reading data in order).
- A leaf node usually stores 2 to 4 key values.
- Non-leaf (internal) nodes usually have 3 to 5 child pointers, depending on the order of the tree.
- Any internal node (except the root) must have between n/2 and n children, which helps keep the tree balanced.

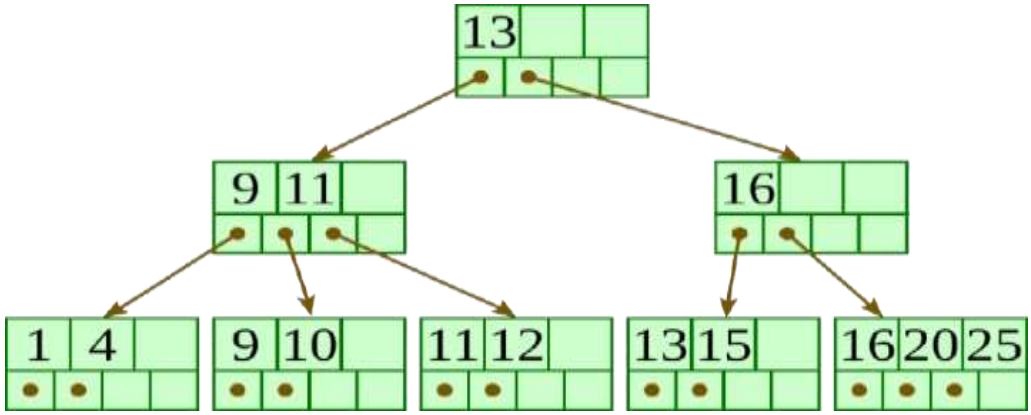


Figure 36: B+ Tree file structure

Indexing:

- Indexing is a data structure technique that allows you to quickly retrieve records from a database file.
- An Index is a small table having only two columns.
- The first column comprises a copy of the primary or candidate key of a table.
- Its second column contains a set of pointers for holding the address of the disk block where that specific key value is stored.
- An index –

Takes a search key as input

Efficiently returns a collection of matching records.

Search Key	Data Reference

Figure 37: Structure of Indexing

Method of Indexing:

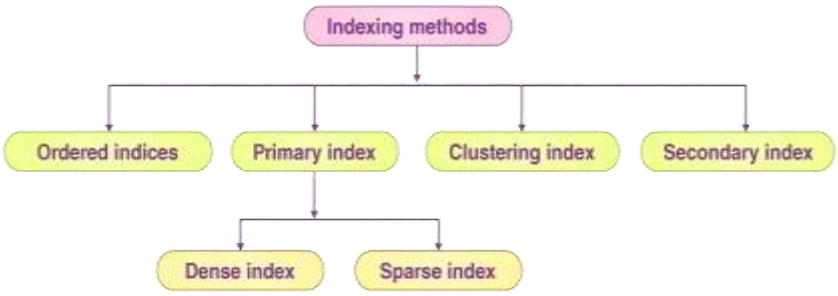


Figure 38: Method of indexing

- **Primary Index**: Primary index is defined on an ordered data file. The data file is ordered on a key field. The key field is generally the primary key of the relation.
- **Secondary Index**: Secondary index may be generated from a field that is a candidate key and has a unique value in every record, or a non-key with duplicate values.
- **Clustering Index**: Clustering index is defined on an ordered data file. The data file is ordered on a non-key field.

Dense Index:

- Each value in the data file has a corresponding index record in the dense index.
- This makes searching faster.
- The number of records in the index is the same as in the main table.
- Extra space is needed to store the index.
- Each index record contains the search key and a pointer to the actual data on the disk.

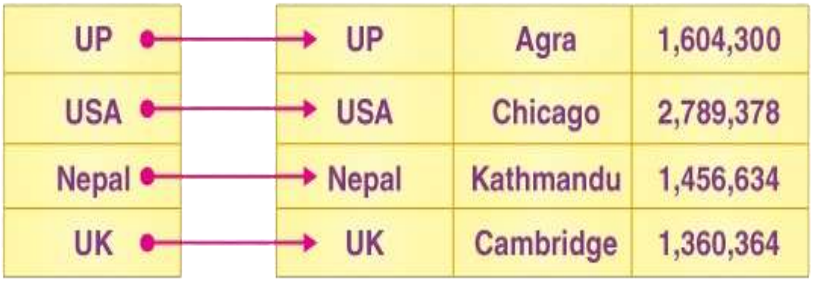


Figure 39: Dense Indexing

Sparse Index

- Not every search key has an index record.

- Each index record has a search key and a pointer to the data on disk.
- To find data, first follow the index to reach an approximate location.
- If the exact data is not at that location, a sequential search is done until it is found.

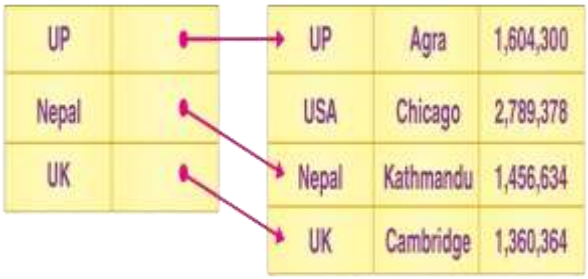


Figure 41: Sparse indexing

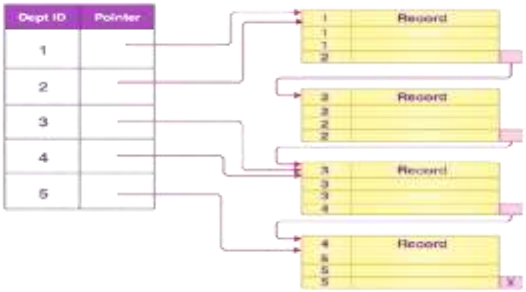


Figure 40: Clustering indexing

Secondary (Multilevel) Index:

- Index records contain search key values and pointers to data.
- Stored on disk along with the actual database files.
- As the database grows, the size of the index also increases.
- Keeping index in main memory speeds up searches.
- A single-level index may be too large for memory, causing multiple disk accesses.

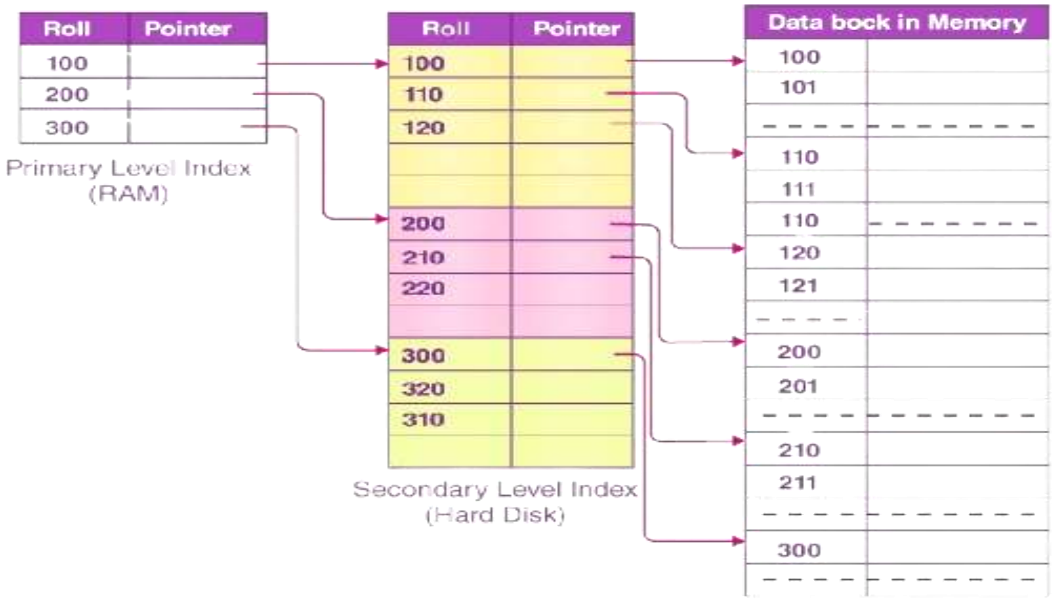


Figure 42: Secondary (multilevel) indexing